3.3 Statements and Functions

Building blocks, not syntax trivia

STATEMENTS

Each instruction we give the computer is a **statement** - executed top to bottom, one after another.

```
greetPerson("Ada");
greetPerson("Grace");
greetPerson("Alan");
```

Three statements. Same function, three different inputs.

A FUNCTION

A named, reusable block of statements - write the logic once, call it as many times as you like.

```
void greetPerson(std::string name) {
    std::println("Hello, {}!", name);
}
```

`name` is a **parameter** - a placeholder filled in by whoever calls it.

THE WHOLE PROGRAM

```cpp
#include <print>
#include <string>

void greetPerson(std::string name) {
    std::println("Hello, {}!", name);
}

int main() {
    greetPerson("Ada");
    greetPerson("Grace");
    greetPerson("Alan");

    return 0;
}
```

COMPILE-TIME VS. RUNTIME

Compile-time error

The program won't even build.

```
greetPersonn("Ada");
// typo: no such function
```

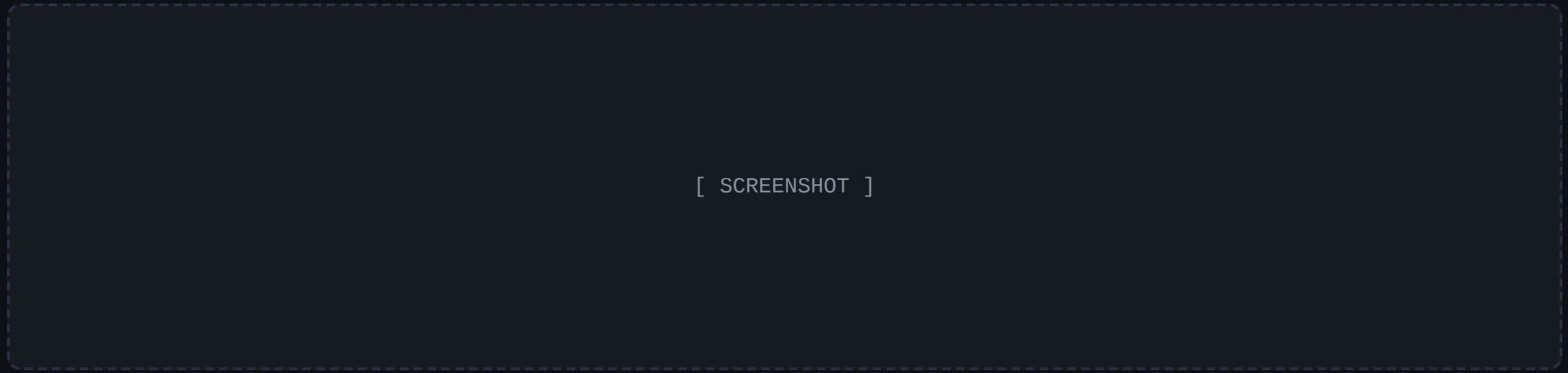
Runtime error

It builds, but breaks while running.

```
int trouble = numerator / 0;
// compiles fine, crashes when run
```

START THE DEBUGGING HABIT NOW

> Even here - before there's any input to read - breakpoint the first line inside `greetPerson` and look at what `name` holds. Get comfortable pausing a program and inspecting a value **before** it gets complicated.



Next: 3.4

Talking to the User